

Systeme d'invitation de compte — Modifications

Backend Requises

Projet : Holy Fork — holly_pi (Django REST Framework) **Date** : 2026-07-19 **Destinataire** : Developpeur backend **Priorite** : P0 (bloquant) → P1 (fonctionnel) → P2 (UX)

Resume executif

Le frontend a implemente un systeme d'invitation d'employes permettant a un gerant de creer un compte utilisateur (User + Employe + RestaurantEmploye) en un seul appel. Le flow fonctionne techniquement mais presente des failles de securite critiques et des donnees perdues cote backend.

Etat actuel : Le frontend appelle `POST /api/settings/users/` avec les champs suivants :

- username, email, password, password2
- first_name, last_name, employee_first_name, employee_last_name
- pin_code, type_employe_id, restaurant_id
- **salary, hire_date, phone_number** (envoyes mais ignorees par le backend)

Problemes identifies :

1. Aucune verification de permission — n'importe quel user authentifie peut creer des comptes
 2. Les champs salary, hire_date, phone_number sont ignorees → employe cree avec des valeurs par default
 3. Le GET ne filtre pas par restaurant → expose tous les users du systeme
 4. Impossible de desactiver un compte (is_active est read-only)
 5. Pas d'envoi d'email d'invitation
 6. Pas de reset/change password
-

P0 — Securite critique (a deployer immediatement)

1. Ajouter la permission `MANAGE_STAFF` sur `POST /api/settings/users/`

Fichier : `apps/settings/views.py` — classe `UsersSettingsView` (ligne 86)

Probleme : La vue utilise uniquement `permission_classes = [IsAuthenticated]`. Un serveur ou un stagiaire peut creer un compte gerant via un simple curl.

Impact securite : Escalade de privileges. Tout user authentifie peut se donner les droits admin.

Implementation :

```
# apps/settings/views.py — REMPLACER lignes 86-104

from apps.staff.permissions import HasPermission
from apps.staff.employee_roles import Permission

@extend_schema_view(
    get=extend_schema(
        operation_id='settings_users_list',
```

```

        parameters=[OpenApiParameter(name='restaurant_id', type=int,
required=False)],
        responses={200: UserProfileSerializer},
    ),
    post=extend_schema(
        operation_id='settings_users_create',
        request=UserRegistrationSerializer,
        responses={201: UserProfileSerializer, 400: None, 403: None},
    ),
)
class UsersSettingsView(APIView):
    """
    GET /api/settings/users?restaurant_id=X - liste des utilisateurs
    POST /api/settings/users - creation utilisateur (requiert MANAGE_STAFF)
    """
    permission_classes = [IsAuthenticated]
    required_permission = Permission.MANAGE_STAFF

    def get_permissions(self):
        """GET = IsAuthenticated, POST = IsAuthenticated + MANAGE_STAFF."""
        if self.request.method == 'POST':
            return [IsAuthenticated(), HasPermission()]
        return [IsAuthenticated()]

    def get(self, request):
        restaurant_id = request.query_params.get('restaurant_id')
        if restaurant_id:
            from apps.staff.models import RestaurantEmploye
            employe_ids = RestaurantEmploye.objects.filter(
                restaurant_id=restaurant_id
            ).values_list('employe__user_id', flat=True)
            users = HollyUser.objects.filter(id__in=employe_ids)
        else:
            users = HollyUser.objects.all()
        serializer = UserProfileSerializer(users, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = UserRegistrationSerializer(
            data=request.data,
            context={'request': request, 'is_admin_invite': True}
        )
        if serializer.is_valid():
            user = serializer.save()
            return Response(UserProfileSerializer(user).data, status=201)
        return Response(serializer.errors, status=400)

```

Verification : Apres deploiement, tester avec un token d'un user role "Serveur" → doit retourner 403.

2. Rendre restaurant_id optionnel et accepter salary/hire_date/phone_number

Fichier : apps/authentication/serializers.py — classe UserRegistrationSerializer (ligne 22)

Probleme :

- `restaurant_id` est `required=True` → impossible de creer un employe sans l'assigner
- `salary` , `hire_date` , `phone_number` ne sont pas dans les fields → ignores silencieusement
- L'employe est cree avec `salaire=0`, `date_embauche=today`, `telephone=null` meme si le frontend envoie les bonnes valeurs

Implementation :

```
# apps/authentication/serializers.py – REMPLACER la classe
UserRegistrationSerializer

from decimal import Decimal
from django.utils import timezone

class UserRegistrationSerializer(serializers.ModelSerializer):
    """
    Serializer pour l'inscription d'un nouvel utilisateur avec creation d'employe et
    PIN code.
    Utilise par :
    - POST /api/auth/register (auto-inscription, restaurant_id obligatoire)
    - POST /api/settings/users/ (invitation admin, restaurant_id optionnel)
    """
    email = serializers.EmailField(
        required=True,
        validators=[UniqueValidator(queryset=User.objects.all())]
    )
    password = serializers.CharField(
        write_only=True,
        required=True,
        validators=[validate_password]
    )
    password2 = serializers.CharField(write_only=True, required=True)

    # Employee identity
    employee_last_name = serializers.CharField(required=True, max_length=100)
    employee_first_name = serializers.CharField(required=True, max_length=100)
    pin_code = serializers.CharField(
        required=True,
        max_length=4,
        min_length=4,
        validators=[RegexValidator(r'^\d{4}$', 'Le code PIN doit contenir exactement
4 chiffres.')]
    )
    type_employe_id = serializers.PrimaryKeyRelatedField(
        queryset=TypeEmploye.objects.all(),
        required=True,
        write_only=True
    )

    # Restaurant – optionnel pour les invitations admin
```

```

restaurant_id = serializers.PrimaryKeyRelatedField(
    queryset=Restaurant.objects.all(),
    required=False,          # <-- CHANGEMENT (etait True)
    allow_null=True,         # <-- AJOUT
    write_only=True
)

# Employee details – optionnels, pour que l'invitation passe les infos completes
salary = serializers.DecimalField(
    max_digits=10, decimal_places=2,
    required=False, write_only=True, default=Decimal('0.00')
)

hire_date = serializers.DateField(required=False, write_only=True)
phone_number = serializers.CharField(
    max_length=20, required=False, write_only=True, allow_blank=True
)

class Meta:
    model = User
    fields = (
        'username', 'email', 'password', 'password2',
        'first_name', 'last_name',
        'employee_first_name', 'employee_last_name',
        'pin_code', 'type_employe_id', 'restaurant_id',
        'salary', 'hire_date', 'phone_number',
    )

    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError({
                "password": "Les mots de passe ne correspondent pas."
            })

        # Validate PIN code uniqueness within restaurant (only if restaurant
        # provided)
        pin_code = attrs.get('pin_code')
        restaurant = attrs.get('restaurant_id')

        if restaurant and pin_code:
            existing_restaurant_employes = RestaurantEmploye.objects.filter(
                restaurant=restaurant
            ).select_related('employe')

            for re in existing_restaurant_employes:
                if re.employe.pin_code and re.employe.pin_code == pin_code:
                    raise serializers.ValidationError({
                        "pin_code": "Ce PIN code est deja utilise par un autre
employé dans ce restaurant."
                    })

        # Si pas de restaurant mais contexte d'auto-inscription, exiger
        restaurant_id

```

```

if not restaurant and not self.context.get('is_admin_invite'):
    raise serializers.ValidationError({
        "restaurant_id": "Ce champ est obligatoire pour l'auto-inscription."
    })

return attrs

def create(self, validated_data):
    validated_data.pop('password2', None)
    employee_last_name = validated_data.pop('employee_last_name')
    employee_first_name = validated_data.pop('employee_first_name')
    pin_code = validated_data.pop('pin_code')
    type_employe = validated_data.pop('type_employe_id')
    restaurant = validated_data.pop('restaurant_id', None)
    salary = validated_data.pop('salary', Decimal('0.00'))
    hire_date = validated_data.pop('hire_date', None)
    phone_number = validated_data.pop('phone_number', '')

    user = User.objects.create_user(
        username=validated_data['username'],
        email=validated_data['email'],
        password=validated_data['password'],
        first_name=validated_data.get('first_name', ''),
        last_name=validated_data.get('last_name', '')
    )

    employe = Employee.objects.create(
        user=user,
        nom=employee_last_name,
        prenom=employee_first_name,
        pin_code=pin_code,
        type_employe=type_employe,
        salaire=salary,
        date_embauche=hire_date or timezone.localtime(),
        numero_telephone=phone_number or '',
    )

    if restaurant:
        RestaurantEmployee.objects.create(
            restaurant=restaurant,
            employe=employe
        )

    return user

```

Impact sur /api/auth/register : L'auto-inscription (sans `is_admin_invite` dans le context) continuera a exiger `restaurant_id` grace a la validation conditionnelle dans `validate()` .

Verification :

1. POST `/api/settings/users/` avec `restaurant_id` + `salary` + `hire_date` → verifie que l'employe est cree avec les bonnes valeurs

2. POST /api/settings/users/ sans `restaurant_id` → vérifie que l'employé est créé sans RestaurantEmploye
 3. POST /api/auth/register sans `restaurant_id` → doit retourner 400
-

P1 — Fonctionnalités manquantes

3. Filtrer GET /api/settings/users/ par restaurant

Deja inclus dans le code de la section 1. Le GET filtre maintenant par `restaurant_id` quand le parametre est fourni.

Amelioration supplementaire (optionnelle) : Limiter aux restaurants accessibles par l'utilisateur :

```
# Version plus securisee du GET
def get(self, request):
    from apps.staff.permissions_utils import PermissionChecker
    from apps.staff.models import RestaurantEmploye

    restaurant_id = request.query_params.get('restaurant_id')

    if restaurant_id:
        # Verifier que le demandeur a acces a ce restaurant
        checker = PermissionChecker(user=request.user)
        if not checker.has_access_to_restaurant(int(restaurant_id)):
            return Response({'error': 'Acces refuse a ce restaurant'}, status=403)

        employe_ids = RestaurantEmploye.objects.filter(
            restaurant_id=restaurant_id
        ).values_list('employe__user_id', flat=True)
        users = HollyUser.objects.filter(id__in=employe_ids)
    else:
        # Sans filtre : retourner les users des restaurants accessibles
        checker = PermissionChecker(user=request.user)
        from apps.staff.employee_roles import EmployeeRole
        if checker.role == EmployeeRole.SUPER_ADMIN_GROUPE:
            users = HollyUser.objects.all()
        else:
            # Recuperer les restaurants accessibles
            my_restaurants = RestaurantEmploye.objects.filter(
                employe__user=request.user
            ).values_list('restaurant_id', flat=True)
            employe_ids = RestaurantEmploye.objects.filter(
                restaurant_id__in=my_restaurants
            ).values_list('employe__user_id', flat=True)
            users = HollyUser.objects.filter(id__in=employe_ids)

    serializer = UserProfileSerializer(users, many=True)
    return Response(serializer.data)
```

4. Rendre is_active writable pour desactiver un compte

Fichier : apps/authentication/serializers.py

Probleme : UserProfileSerializer a is_active en read_only_fields . Un gerant ne peut pas desactiver le compte d'un employe qui quitte le restaurant.

Implementation : Creer un serializer dedie pour les admins :

```
# apps/authentication/serializers.py – AJOUTER apres UserProfileSerializer

class AdminUserUpdateSerializer(serializers.ModelSerializer):
    """
    Serializer pour la mise a jour admin d'un utilisateur.
    Permet de modifier email, noms, et is_active.
    Reserve aux utilisateurs avec permission MANAGE_STAFF.
    """
    class Meta:
        model = User
        fields = ('id', 'username', 'email', 'first_name', 'last_name', 'is_active')
        read_only_fields = ('id', 'username')
```

Modifier la vue PATCH :

```
# apps/settings/views.py – classe UserSettingsDetailView

from apps.staff.permissions import HasPermission
from apps.staff.employee_roles import Permission
from apps.authentication.serializers import AdminUserUpdateSerializer

class UserSettingsDetailView(APIView):
    """
    GET /api/settings/users/{user_id}/ – detail d'un utilisateur
    PATCH /api/settings/users/{user_id}/ – mise a jour (requiert MANAGE_STAFF)
    """
    permission_classes = [IsAuthenticated]
    required_permission = Permission.MANAGE_STAFF

    def get_permissions(self):
        if self.request.method == 'PATCH':
            return [IsAuthenticated(), HasPermission()]
        return [IsAuthenticated()]

    def get(self, request, user_id):
        try:
            user = HollyUser.objects.get(id=user_id)
            return Response(UserProfileSerializer(user).data)
        except HollyUser.DoesNotExist:
            return Response({'error': 'Utilisateur introuvable'}, status=404)

    def patch(self, request, user_id):
        try:
```

```

        user = HollyUser.objects.get(id=user_id)
        serializer = AdminUserUpdateSerializer(user, data=request.data,
partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(UserProfileSerializer(user).data)
        return Response(serializer.errors, status=400)
    except HollyUser.DoesNotExist:
        return Response({'error': 'Utilisateur introuvable'}, status=404)

```

Verification :

- PATCH /api/settings/users/5/ avec {"is_active": false} par un gerant → 200
- PATCH /api/settings/users/5/ par un serveur → 403
- Login avec le user desactive → doit retourner "Compte utilisateur desactive"

5. Ajouter l'email dans la reponse GET /api/employees/

Fichier : apps/staff/serializers.py — classe EmployeeSerializer

Probleme : La reponse Employee contient user_id mais pas l'email. Le frontend ne peut pas afficher l'email d'un employe ayant un compte.

Implementation :

```

# apps/staff/serializers.py — MODIFIER EmployeeSerializer

class EmployeeSerializer(serializers.HyperlinkedModelSerializer):
    last_name = serializers.CharField(source='nom', max_length=100)
    first_name = serializers.CharField(source='prenom', max_length=100)
    salary = serializers.DecimalField(source='salaire', max_digits=10,
decimal_places=2, required=False, default=0)
    hire_date = serializers.DateField(source='date_embauche', required=False)
    phone_number = serializers.CharField(source='numero_telephone', max_length=20,
required=False, allow_blank=True)
    type_employe_id = serializers.PrimaryKeyRelatedField(
        queryset=TypeEmploye.objects.all(),
        source='type_employe',
    )
    type_employe_name = serializers.CharField(
        source='type_employe.nom_type', read_only=True
    )
    user_id = serializers.PrimaryKeyRelatedField(
        queryset=User.objects.all(),
        source='user',
        required=False
    )
    # AJOUT : email du user lie (read-only)
    email = serializers.SerializerMethodField()

    class Meta:
        model = Employee

```



```

fields = [
    'id',
    'user_id',
    'last_name',
    'first_name',
    'type_employe_id',
    'type_employe_name',
    'salary',
    'hire_date',
    'phone_number',
    'email', # <-- AJOUT
]
extra_kwargs = {
    'id': {'read_only': True},
}

def get_email(self, obj):
    """Retourne l'email du user associe, ou None si pas de compte."""
    return obj.user.email if obj.user else None

# ... reste du code inchangé (validate, create, update)

```

6. Validation hierarchique sur changement de role

Fichier : `apps/staff/serializers.py` — methode `update` de `EmployeeSerializer`

Probleme : N'importe quel user avec acces au PATCH peut changer le `type_employe_id` d'un employe, y compris se promouvoir lui-meme.

Implementation :

```

# apps/staff/serializers.py — MODIFIER la methode update de EmployeeSerializer

from apps.staff.employee_roles import EmployeeRole, is_higher_role
from apps.staff.permissions_utils import PermissionChecker

def update(self, instance, validated_data):
    # Validation hierarchie sur changement de role
    new_type = validated_data.get('type_employe')
    if new_type and new_type != instance.type_employe:
        request = self.context.get('request')
        if request and request.user:
            checker = PermissionChecker(user=request.user)
            try:
                new_role = EmployeeRole(new_type.nom_type)
                if not is_higher_role(checker.role, new_role):
                    raise serializers.ValidationError({
                        "type_employe_id": "Impossible d'assigner un role superieur
ou egal au votre."
                    })
            except:

```

```

        except ValueError:
            pass # Role non reconnu dans l'enum, laisser passer

    if 'nom' in validated_data:
        instance.nom = validated_data['nom']
    if 'prenom' in validated_data:
        instance.prenom = validated_data['prenom']
    if 'salaire' in validated_data:
        instance.salaire = validated_data['salaire']
    if 'date_embauche' in validated_data:
        instance.date_embauche = validated_data['date_embauche']
    if 'numero_telephone' in validated_data:
        instance.numero_telephone = validated_data['numero_telephone']
    if 'type_employe' in validated_data:
        instance.type_employe = validated_data['type_employe']
    if 'user' in validated_data:
        instance.user = validated_data['user']
    instance.save()
    return instance

```

P2 — Experience utilisateur

7. Endpoints de gestion de mot de passe

Fichier : `apps/authentication/views.py` + `apps/authentication/urls.py`

Les employes invites recoivent un mot de passe temporaire. Ils doivent pouvoir le changer.

7a. POST `/api/auth/change-password/` (user connecte)

```

# apps/authentication/views.py – AJOUTER

from rest_framework.permissions import IsAuthenticated
from django.contrib.auth.password_validation import validate_password

class ChangePasswordView(APIView):
    """Permet a un user connecte de changer son mot de passe."""
    permission_classes = [IsAuthenticated]

    @extend_schema(
        request={
            'type': 'object',
            'properties': {
                'old_password': {'type': 'string'},
                'new_password': {'type': 'string'},
                'new_password2': {'type': 'string'},
            },
            'required': ['old_password', 'new_password', 'new_password2'],
        },
        responses={200: None, 400: None},
    )

```

```

)
def post(self, request):
    old_password = request.data.get('old_password')
    new_password = request.data.get('new_password')
    new_password2 = request.data.get('new_password2')

    if not request.user.check_password(old_password):
        return Response(
            {'old_password': ['Mot de passe actuel incorrect.']},
            status=400
        )

    if new_password != new_password2:
        return Response(
            {'new_password': ['Les mots de passe ne correspondent pas.']},
            status=400
        )

    try:
        validate_password(new_password, request.user)
    except Exception as e:
        return Response({'new_password': list(e.messages)}, status=400)

    request.user.set_password(new_password)
    request.user.save()
    return Response({'message': 'Mot de passe modifie avec succes.'})

```

7b. POST /api/auth/forgot-password/ (non connecte)

```

# apps/authentication/views.py – AJOUTER

from django.contrib.auth.tokens import default_token_generator
from django.utils.http import urlsafe_base64_encode
from django.utils.encoding import force_bytes
from django.core.mail import send_mail
from django.template.loader import render_to_string
from django.conf import settings

class ForgotPasswordView(APIView):
    """Envoie un email de reset password si le compte existe."""
    permission_classes = [] # Public
    authentication_classes = []

    def post(self, request):
        email = request.data.get('email', '').strip().lower()
        # Toujours retourner 200 pour ne pas reveler l'existence d'un compte
        response = {'message': 'Si un compte existe avec cet email, un lien de
reinitialisation a ete envoye.'}

        if not email:

```

```

        return Response(response)

    try:
        user = User.objects.get(email=email, is_active=True)
    except User.DoesNotExist:
        return Response(response)

    # Generer le token
    token = default_token_generator.make_token(user)
    uid = urlsafe_base64_encode(force_bytes(user.pk))
    reset_url = f'{settings.FRONTEND_URL}/reset-password?uid={uid}&token={token}'

    # Envoyer l'email
    try:
        html = render_to_string('emails/reset_password.html', {
            'name': user.get_full_name() or user.username,
            'reset_url': reset_url,
        })
        send_mail(
            subject='Holy Fork – Reinitialisation de mot de passe',
            message=f'Cliquez sur ce lien pour reinitialiser votre mot de passe : {reset_url}',
            html_message=html,
            from_email=settings.DEFAULT_FROM_EMAIL,
            recipient_list=[user.email],
            fail_silently=True,
        )
    except Exception:
        pass # Ne pas exposer les erreurs d'envoi

    return Response(response)

```

7c. POST /api/auth/reset-password/ (avec token)

```

# apps/authentication/views.py – AJOUTER

from django.utils.http import urlsafe_base64_decode

class ResetPasswordView(APIView):
    """Confirme le reset de mot de passe avec le token envoye par email."""
    permission_classes = []
    authentication_classes = []

    def post(self, request):
        uid = request.data.get('uid')
        token = request.data.get('token')
        new_password = request.data.get('new_password')
        new_password2 = request.data.get('new_password2')

```

```

if not all([uid, token, new_password, new_password2]):
    return Response({'error': 'Tous les champs sont requis.'}, status=400)

if new_password != new_password2:
    return Response({'new_password': ['Les mots de passe ne correspondent
pas.']}}, status=400)

try:
    user_id = urlsafe_base64_decode(uid).decode()
    user = User.objects.get(pk=user_id)
except (TypeError, ValueError, User.DoesNotExist):
    return Response({'error': 'Lien invalide ou expire.'}, status=400)

if not default_token_generator.check_token(user, token):
    return Response({'error': 'Lien invalide ou expire.'}, status=400)

try:
    validate_password(new_password, user)
except Exception as e:
    return Response({'new_password': list(e.messages)}, status=400)

user.set_password(new_password)
user.save()
return Response({'message': 'Mot de passe reinitialise avec succes.'})

```

7d. Enregistrer les URLs

```

# apps/authentication/urls.py – AJOUTER dans urlpatterns

from .views import ChangePasswordView, ForgotPasswordView, ResetPasswordView

urlpatterns = [
    # ... URLs existantes ...
    path('change-password/', ChangePasswordView.as_view(), name='change-password'),
    path('forgot-password/', ForgotPasswordView.as_view(), name='forgot-password'),
    path('reset-password/', ResetPasswordView.as_view(), name='reset-password'),
]

```

8. Configuration email

Fichier : `holly_pi/settings.py`

```

# AJOUTER dans settings.py

# Email configuration
EMAIL_BACKEND = os.environ.get(
    'EMAIL_BACKEND',
    'django.core.mail.backends.console.EmailBackend' # Console en dev
)
EMAIL_HOST = os.environ.get('EMAIL_HOST', 'smtp.sendgrid.net')

```

```
EMAIL_PORT = int(os.environ.get('EMAIL_PORT', 587))
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.environ.get('EMAIL_HOST_USER', '')
EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_HOST_PASSWORD', '')
DEFAULT_FROM_EMAIL = os.environ.get('DEFAULT_FROM_EMAIL', 'Holy Fork
<noreply@hollyfork.fr>')
FRONTEND_URL = os.environ.get('FRONTEND_URL', 'http://localhost:5173')
```

Variables d'environnement pour la production :

```
EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend
EMAIL_HOST=smtp.sendgrid.net
EMAIL_PORT=587
EMAIL_HOST_USER=apikey
EMAIL_HOST_PASSWORD=SG.xxxxxx
DEFAULT_FROM_EMAIL=Holy Fork <noreply@hollyfork.fr>
FRONTEND_URL=https://app.hollyfork.fr
```

9. Email d'invitation automatique

Fichier : `apps/authentication/serializers.py` — methode `create` de `UserRegistrationSerializer`

Ajouter a la fin de la methode `create()` :

```
def create(self, validated_data):
    # ... (code existant de creation user + employe + restaurant_employe) ...

    # Envoyer l'email d'invitation si c'est une creation admin
    if self.context.get('is_admin_invite'):
        self._send_invitation_email(user, validated_data.get('password', ''))

    return user

def _send_invitation_email(self, user, password):
    """Envoie un email d'invitation avec les credentials."""
    from django.core.mail import send_mail
    from django.template.loader import render_to_string
    from django.conf import settings

    try:
        html = render_to_string('emails/invitation.html', {
            'name': user.get_full_name() or user.username,
            'email': user.email,
            'password': password,
            'login_url': f"{settings.FRONTEND_URL}/login",
        })
        send_mail(
            subject='Bienvenue sur Holy Fork – Vos identifiants de connexion',
            message=(
                f"Bonjour {user.get_full_name()},\n\n"
```

```

        f"Un compte Holy Fork a ete cree pour vous.\n"
        f"Email : {user.email}\n"
        f"Mot de passe : {password}\n\n"
        f"Connectez-vous sur {settings.FRONTEND_URL}/login\n"
        f"Pensez a changer votre mot de passe a la premiere connexion."
    ),
    html_message=html,
    from_email=settings.DEFAULT_FROM_EMAIL,
    recipient_list=[user.email],
    fail_silently=True, # Ne pas bloquer la creation si l'email echoue
)
except Exception:
    pass # Log l'erreur mais ne pas faire echouer la creation

```

Template email : Creer `apps/authentication/templates/emails/invitation.html`

```

<!DOCTYPE html>
<html>
<head><meta charset="utf-8"></head>
<body style="font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
padding: 40px 20px; background: #f9fafb;">
    <div style="max-width: 480px; margin: 0 auto; background: white; border-radius:
12px; padding: 32px; box-shadow: 0 1px 3px rgba(0,0,0,0.1);">
        <h1 style="font-size: 20px; margin: 0 0 16px 0;">Bienvenue sur Holy Fork</h1>
        <p style="color: #374151; line-height: 1.6;">
            Bonjour {{ name }},<br><br>
            Un compte a ete cree pour vous. Voici vos identifiants de connexion :
        </p>
        <div style="background: #f3f4f6; border-radius: 8px; padding: 16px; margin: 20px
0; font-family: monospace;">
            <div><strong>Email :</strong> {{ email }}</div>
            <div style="margin-top: 8px;"><strong>Mot de passe :</strong> {{ password }}
        </div>
        </div>
        <p style="color: #6b7280; font-size: 14px;">
            Pensez a changer votre mot de passe a la premiere connexion.
        </p>
        <a href="{{ login_url }}" style="display: inline-block; background: #f97316;
color: white; text-decoration: none; padding: 12px 24px; border-radius: 8px; margin-
top: 16px; font-weight: 500;">
            Se connecter
        </a>
    </div>
</body>
</html>

```

Creer aussi `apps/authentication/templates/emails/reset_password.html` :

```

<!DOCTYPE html>
<html>
<head><meta charset="utf-8"></head>

```

```

<body style="font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
padding: 40px 20px; background: #f9fafb;">
  <div style="max-width: 480px; margin: 0 auto; background: white; border-radius:
12px; padding: 32px; box-shadow: 0 1px 3px rgba(0,0,0,0.1);">
    <h1 style="font-size: 20px; margin: 0 0 16px;">Reinitialisation de mot de
passe</h1>
    <p style="color: #374151; line-height: 1.6;">
      Bonjour {{ name }},<br><br>
      Vous avez demande la reinitialisation de votre mot de passe Holy Fork.
      Cliquez sur le bouton ci-dessous pour choisir un nouveau mot de passe.
    </p>
    <a href="{{ reset_url }}" style="display: inline-block; background: #f97316;
color: white; text-decoration: none; padding: 12px 24px; border-radius: 8px; margin-
top: 16px; font-weight: 500;">
      Reinitialiser mon mot de passe
    </a>
    <p style="color: #9ca3af; font-size: 12px; margin-top: 24px;">
      Si vous n'avez pas fait cette demande, ignorez cet email. Le lien expire dans
24h.
    </p>
  </div>
</body>
</html>

```

Ordre d'implementation recommande

Etape	Fichier(s)	Temps estime	Priorite
1	apps/settings/views.py — permission POST + filtrage GET	15 min	P0
2	apps/authentication/serializers.py — restaurant_id optionnel + salary/hire_date/phone	20 min	P0
3	Tester POST /api/settings/users/ depuis le frontend	5 min	P0
4	apps/authentication/serializers.py — AdminUserUpdateSerializer	5 min	P1
5	apps/settings/views.py — PATCH avec is_active	10 min	P1
6	apps/staff/serializers.py — email dans EmployeeSerializer + validation hierarchie	15 min	P1
7	apps/authentication/views.py — change-password	10 min	P2
8	apps/authentication/views.py — forgot-password + reset-password	20 min	P2
9	holly_pi/settings.py — config email	5 min	P2
10	Templates email + envoi dans create()	15 min	P2

Total estime : ~2h de travail backend.

Tests a ecrire

```
# apps/settings/tests.py ou apps/authentication/tests.py

class InvitationSystemTests(APITestCase):
    """Tests du systeme d'invitation."""

    def setUp(self):
        # Creer un gerant (a MANAGE_STAFF)
        self.manager_type = TypeEmploye.objects.create(nom_type="Admin
Etablissement")
        self.server_type = TypeEmploye.objects.create(nom_type="Serveur")
        self.restaurant = Restaurant.objects.create(nom_restaurant="Test Resto",
...)

        self.manager_user = User.objects.create_user(username="manager",
password="pass", email="m@t.com")
        self.manager_emp = Employe.objects.create(user=self.manager_user, nom="M",
prenom="M", type_employe=self.manager_type)
        RestaurantEmploye.objects.create(restaurant=self.restaurant,
employe=self.manager_emp)

        self.server_user = User.objects.create_user(username="server",
password="pass", email="s@t.com")
        self.server_emp = Employe.objects.create(user=self.server_user, nom="S",
prenom="S", type_employe=self.server_type)
        RestaurantEmploye.objects.create(restaurant=self.restaurant,
employe=self.server_emp)

    def test_manager_can_invite(self):
        """Un gerant peut creer un compte via POST /api/settings/users/."""
        self.client.force_authenticate(user=self.manager_user)
        resp = self.client.post('/api/settings/users/', {
            'username': 'newuser',
            'email': 'new@test.com',
            'password': 'SecurePass123!',
            'password2': 'SecurePass123!',
            'employee_first_name': 'Jean',
            'employee_last_name': 'Dupont',
            'pin_code': '1234',
            'type_employe_id': self.server_type.id,
            'restaurant_id': self.restaurant.id_restaurant,
            'salary': '2500.00',
            'hire_date': '2026-07-19',
            'phone_number': '+33612345678',
        })
        self.assertEqual(resp.status_code, 201)
        # Verifier que l'employe a le bon salaire
        emp = Employe.objects.get(user__email='new@test.com')
        self.assertEqual(emp.salaire, Decimal('2500.00'))
```

```

self.assertEqual(emp.numero_telephone, '+33612345678')

def test_server_cannot_invite(self):
    """Un serveur ne peut pas creer de compte (403)."""
    self.client.force_authenticate(user=self.server_user)
    resp = self.client.post('/api/settings/users/', {...})
    self.assertEqual(resp.status_code, 403)

def test_invite_without_restaurant(self):
    """Invitation sans restaurant_id cree l'employe sans assignment."""
    self.client.force_authenticate(user=self.manager_user)
    resp = self.client.post('/api/settings/users/', {
        'username': 'norest',
        'email': 'norest@test.com',
        'password': 'SecurePass123!',
        'password2': 'SecurePass123!',
        'employee_first_name': 'Sans',
        'employee_last_name': 'Restaurant',
        'pin_code': '5678',
        'type_employe_id': self.server_type.id,
        # PAS de restaurant_id
    })
    self.assertEqual(resp.status_code, 201)
    emp = Employee.objects.get(user__email='norest@test.com')
    self.assertFalse(RestaurantEmployee.objects.filter(employe=emp).exists())

def test_deactivate_account(self):
    """Un gerant peut desactiver un compte."""
    self.client.force_authenticate(user=self.manager_user)
    resp = self.client.patch(f'/api/settings/users/{self.server_user.id}/', {
        'is_active': False
    })
    self.assertEqual(resp.status_code, 200)
    self.server_user.refresh_from_db()
    self.assertFalse(self.server_user.is_active)

def test_change_password(self):
    """Un user connecte peut changer son mot de passe."""
    self.client.force_authenticate(user=self.server_user)
    resp = self.client.post('/api/auth/change-password/', {
        'old_password': 'pass',
        'new_password': 'NewSecure456!',
        'new_password2': 'NewSecure456!',
    })
    self.assertEqual(resp.status_code, 200)
    self.server_user.refresh_from_db()
    self.assertTrue(self.server_user.check_password('NewSecure456!'))

```

Schema de la base de donnees impliquee

```

HollyUser (auth_user)
├─ id, username, email, password, first_name, last_name, is_active
└─
  └─ Employee (1:1 via user FK)
      ├─ id, nom, prenom, pin_code, type_employe_id, salaire, date_embauche,
      │   numero_telephone
      └─ RestaurantEmploye (N:M via table de liaison)
          ├─ id, restaurant_id, employe_id
          └─ unique_together: (restaurant, employe)

```

Flow complet de l'invitation :

Frontend	Backend
POST /api/settings/users/	
{email, password, password2,	
firstName, lastName,	
employeeFirstName/LastName,	
pinCode, typeEmployeId,	
restaurantId, salary,	
hireDate, phoneNumber}	
→	
	1. Check IsAuthenticated ✓
	2. Check HasPermission(MANAGE_STAFF) ✓
	3. Validate (email unique, PIN
unique/restaurant, passwords match)	
	4. Create HollyUser
	5. Create Employe (avec salary, hire_date,
phone)	
	6. Create RestaurantEmploye (si restaurant_id
fourni)	
	7. Send invitation email (async,
fail_silently)	
201 {id, username, email,	
first_name, last_name,	
date_joined, is_active}	
←	
Affiche credentials + PIN	
(email + mdp + pin caisse)	